

Ex.No : 4	Word Embeddings
Date : 21.07.2026	

AIM:

To implement Word Embedding techniques using Word2Vec (CBOW and Skip-Gram) and GloVe, and obtain vector representations of words for Natural Language Processing applications.

WORD EMBEDDING:

Word Embedding is a technique in Natural Language Processing (NLP) that represents words as dense numerical vectors in a continuous vector space. Words with similar meanings are placed close to each other, enabling computers to understand semantic and syntactic relationships between words.

2A. WORD2VEC

Word2Vec is a neural network-based word embedding technique developed by **Google** that converts words into continuous numerical vectors based on their context in a large text corpus. It learns semantic and syntactic relationships between words, so words with similar meanings have similar vector representations.

i. Continuous Bag of Words (CBOW):

Continuous Bag of Words (CBOW) predicts the **target word** using the surrounding context words. It is fast and performs well for frequent words.

ALGORITHM

- Import the required libraries.
- Define a collection of sentences.
- Tokenize the sentences.
- Train the Word2Vec model using **CBOW (sg=0)**.
- Enter a word.
- Display its word embedding vector.
- Display similar words.

PROGRAM:

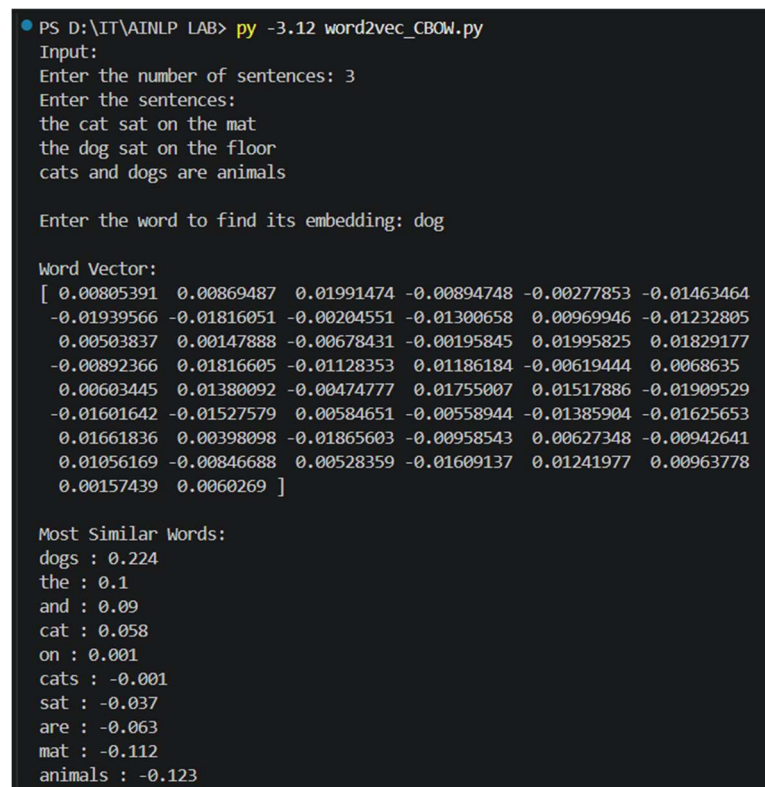
```
from gensim.models import Word2Vec
print("Input:")
n = int(input("Enter the number of sentences: "))
sentences = []
print("Enter the sentences:")
for i in range(n):
```

```

sentence = input().lower().split()
sentences.append(sentence)
model = Word2Vec(sentences, vector_size=50, window=2, min_count=1, sg=0)
word = input("\nEnter the word to find its embedding: ").lower()
if word in model.wv:
    print("\nWord Vector:")
    print(model.wv[word])
    print("\nMost Similar Words:")
    for w, score in model.wv.most_similar(word):
        print(w, ":", round(score, 3))
else:
    print("Word not found in vocabulary.")

```

OUTPUT:



```

PS D:\IT\AINLP LAB> py -3.12 word2vec_CBOW.py
Input:
Enter the number of sentences: 3
Enter the sentences:
the cat sat on the mat
the dog sat on the floor
cats and dogs are animals

Enter the word to find its embedding: dog

Word Vector:
[ 0.00805391 0.00869487 0.01991474 -0.00894748 -0.00277853 -0.01463464
-0.01939566 -0.01816051 -0.00204551 -0.01300658 0.00969946 -0.01232805
0.00503837 0.00147888 -0.00678431 -0.00195845 0.01995825 0.01829177
-0.00892366 0.01816605 -0.01128353 0.01186184 -0.00619444 0.0068635
0.00603445 0.01380092 -0.00474777 0.01755007 0.01517886 -0.01909529
-0.01601642 -0.01527579 0.00584651 -0.00558944 -0.01385904 -0.01625653
0.01661836 0.00398098 -0.01865603 -0.00958543 0.00627348 -0.00942641
0.01056169 -0.00846688 0.00528359 -0.01609137 0.01241977 0.00963778
0.00157439 0.0060269 ]

Most Similar Words:
dogs : 0.224
the : 0.1
and : 0.09
cat : 0.058
on : 0.001
cats : -0.001
sat : -0.037
are : -0.063
mat : -0.112
animals : -0.123

```

ii. Continuous Skip-Gram:

Skip-Gram predicts the **surrounding context words** from the target word. It works better for rare words and produces high-quality embeddings.

ALGORITHM:

- Import the required libraries.
- Define the sentences.
- Train the Word2Vec model using Skip-Gram (sg=1).
- Enter a word.

- Display its embedding vector.
- Display similar words.

PROGRAM:

```

from gensim.models import Word2Vec
print("Input:")
n = int(input("Enter the number of sentences: "))
sentences = []
print("Enter the sentences:")
for i in range(n):
    sentence = input().lower().split()
    sentences.append(sentence)
model = Word2Vec(sentences, vector_size=50, window=2, min_count=1, sg=1)
word = input("\nEnter the word to find its embedding: ").lower()
if word in model.wv:
    print("\nWord Vector:")
    print(model.wv[word])
    print("\nMost Similar Words:")
    for w, score in model.wv.most_similar(word):
        print(w, ":", round(score, 3))
else:
    print("Word not found in vocabulary.")
print()

```

OUTPUT:

```

PS D:\IT\AINLP LAB> py -3.12 word2vec_SKIP_GRAM.py
Input:
Enter the number of sentences: 3
Enter the sentences:
machine learning is interesting
deep learning uses neural networks
artificial intelligence is growing

Enter the word to find its embedding: learning

Word Vector:
[-0.01631583  0.0089916 -0.00827415  0.00164907  0.01699724 -0.00892435
 0.009035   -0.01357392 -0.00709698  0.01879702 -0.00315531  0.00064274
-0.00828126 -0.01536538 -0.00301602  0.00493959 -0.00177605  0.01106732
-0.00548595  0.00452013  0.01091159  0.01669191 -0.00290748 -0.01841629
 0.0087411   0.00114357  0.01488382 -0.00162657 -0.00527683 -0.01750602
-0.00171311  0.00565313  0.01080286  0.01410531 -0.01140624  0.00371764
 0.01217773 -0.0095961  -0.00621452  0.01359526  0.00326295  0.00037983
 0.00694727  0.00043555  0.01923765  0.01012121 -0.01783478 -0.01408312
 0.00180291  0.01278507]

Most Similar Words:
artificial : 0.125
neural : 0.074
is : 0.042
networks : 0.018
growing : 0.011
interesting : 0.001
deep : -0.119
intelligence : -0.174
uses : -0.175
machine : -0.247

```

4B. GLOVE:

GloVe (Global Vectors for Word Representation) is a word embedding technique developed by Stanford University. It learns word vectors using **global word-word co-occurrence statistics**, producing meaningful semantic relationships between words.

ALGORITHM:

- Import the required libraries.
- Download the pre-trained GloVe embeddings.
- Load the embedding model.
- Enter a word.
- Display the embedding vector.
- Display similar words.

PROGRAM:

```
import gensim.downloader as api
print("Loading GloVe model...")
model = api.load("glove-wiki-gigaword-50")
word = input("Enter the word to find its embedding: ").lower()
if word in model:
    print("\nWord Vector:")
    print(model[word])
    print("\nMost Similar Words:")
    for w, score in model.most_similar(word):
        print(w, ":", round(score, 3))
else:
    print("Word not found in vocabulary.")
```

OUTPUT:

```
PS D:\IT\AINLP LAB> py -3.12 glove.py
Loading glove model...
[=====] 100.0% 66.0/66.0MB downloaded

Enter the word to find its embedding: king

Word Vector:
[ 0.50451  0.68607 -0.59517 -0.022801  0.60046 -0.13498 -0.08813
 0.47377 -0.61798 -0.31012 -0.076666  1.493 -0.034189 -0.98173
 0.68229  0.81722 -0.51874 -0.31503 -0.55809  0.66421  0.1961
-0.13495 -0.11476 -0.30344  0.41177 -2.223 -1.0756 -1.0783
-0.34354  0.33505  1.9927 -0.04234 -0.64319  0.71125  0.49159
 0.16754  0.34344 -0.25663 -0.8523  0.1661  0.40102  1.1685
-1.0137 -0.21585 -0.15155  0.78321 -0.91241 -1.6106 -0.64426
-0.51042 ]

Most Similar Words:
prince : 0.824
queen : 0.784
ii : 0.775
emperor : 0.774
son : 0.767
uncle : 0.763
kingdom : 0.754
throne : 0.754
brother : 0.749
ruler : 0.743
```

Problem Understanding & Approach	Implementation & Execution	Code Quality & Documentation	Viva	Time Management	Total
(10)	(15)	(15)	(10)	(10)	(60)

RESULT :

The Word Embedding techniques using Word2Vec (CBOW and Skip-Gram) and GloVe were successfully implemented in Python. The generated word vectors effectively captured semantic relationships, and similar words were retrieved based on their vector representations.

